

How to run capable AI on your own machine, for free)

 charliehills.substack.com/p/how-to-run-claude-code-for-free

Charlie Hills

June 21, 2026

On Friday, June 12th, the US government sent Anthropic a letter.

AI

Anthropic  @AnthropicAI · Jun 13



The US government, citing national security authorities, has issued an export control directive to suspend all access to Fable 5 and Mythos 5 by any foreign national, whether inside or outside the United States, including foreign national Anthropic employees.

The net effect of
[Show more](#)

STATEMENT ON

**The US government
directive to suspend access
to Fable 5 and Mythos 5**

Statement on the US government directive to suspend access to Fable 5 and ...

From anthropic.com

 12K

 54K

 88K

 91M

By 5:21pm Eastern, Fable 5 and Mythos 5 were gone. Not throttled. Switched off, for every customer, everywhere.

Golden hour thinking

Claude Fable 5 is currently unavailable.

[Learn more](#)

Try: draft an email · summarize a doc · plan your week

+

Opus 4.8 High ▾



 Write

 Learn

 From Drive

 From Calendar

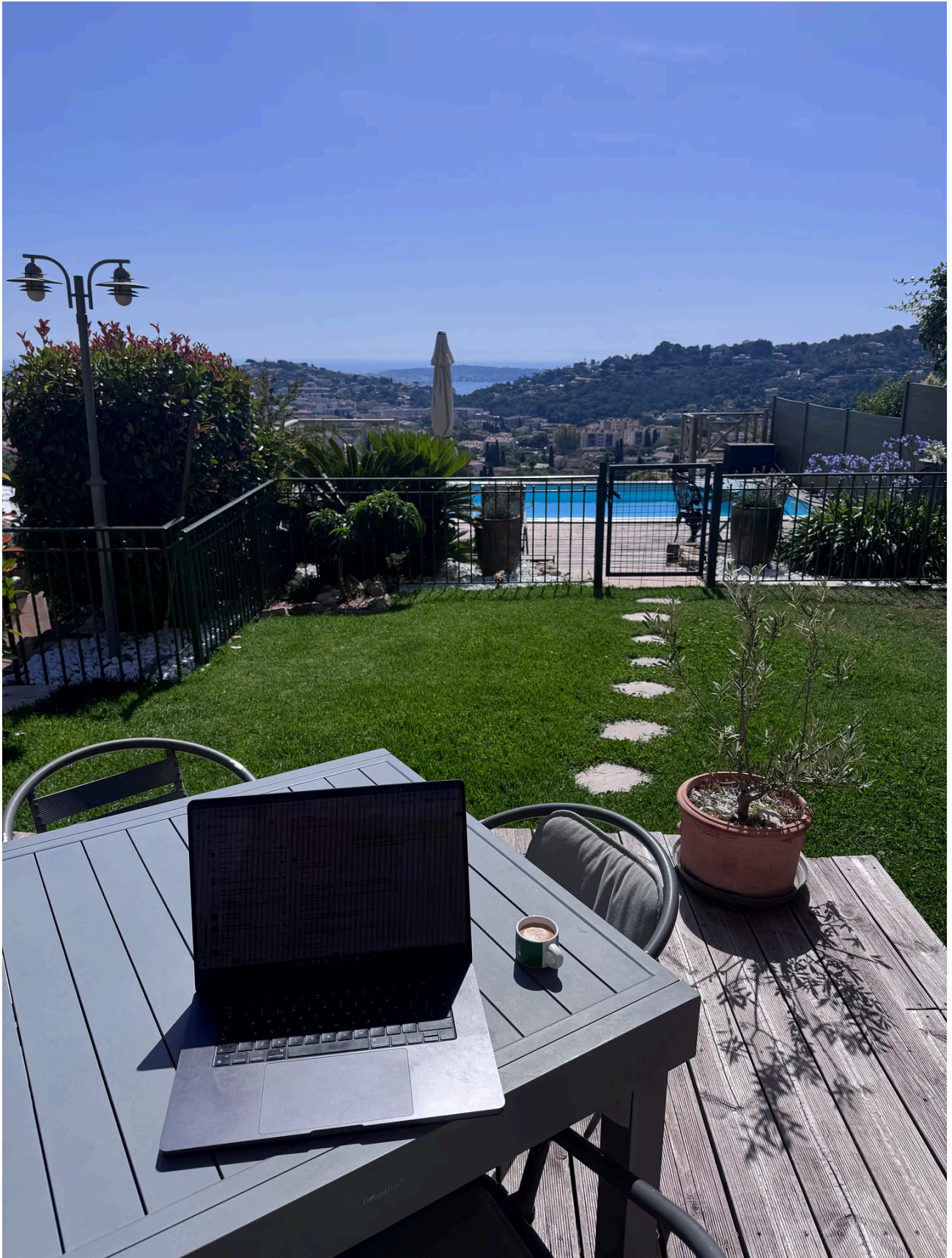
 From Gmail

By the end of this issue you'll have his playbook, plus my own verified setup, so you can run capable AI on your own machine, for free, with no company and no government able to flip the switch.

Have you ever run an AI model locally on your own machine?

Never, wouldn't know where to start

No YouTube video from me this week. I'm writing this from Cannes, coffee in hand and a beautiful view of the bay in front of me. I'll be back in July.



Bienvenue from the Côte d'Azur ☕

If you're new here, welcome. If you've been reading for a while, thank you. Your subscription is what makes issues like this possible.

MarTech AI is a reader-supported publication. To receive new posts and support my work, consider becoming a free or paid subscriber.

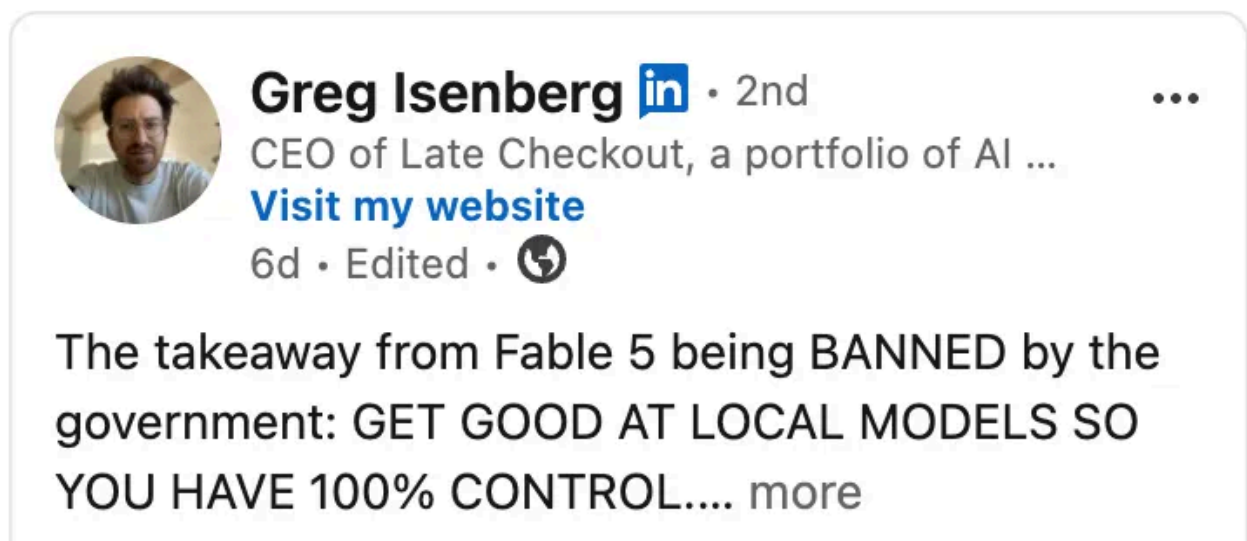
You don't own the model

This was the first time the United States has ever issued an export-control directive for access to an AI model. The order barred foreign nationals from using Fable 5 and Mythos 5, including Anthropic's own staff who aren't US citizens. To comply, Anthropic had to pull both models for everyone.

I'm not here to argue whether the order was right. Plenty of people are doing that already. The lesson sits underneath the politics, and it applies to you whichever model you pay for.

You don't own the model. You rent access to it.

Greg Isenberg put it best, and this whole issue is built on the point he made on LinkedIn this week. His framing: don't build your entire workflow on something that can disappear with one letter. Own part of your stack.



He's seen this film before. He once ran a startup whose biggest traffic source was organic Facebook. The algorithm changed overnight and he lost 99% of it. Anyone who built on rented reach got the same lesson. Email lists exist because people learned it the hard way.

You can already see it happening, and not just with hobbyists. PewDiePie, one of the biggest creators on the planet at 100 million plus subscribers, just went fully local.



MY trillion \$Dollar Project is finally OUT!



3.1m views • 2 weeks ago

He built a 10-GPU rig at home, ran a stack of open models against each other to pick the best answer, and released his own self-hosted AI workspace called Odysseus (<https://github.com/pewdiepie-archdaemon/odysseus>) with his own model in the works.

He owns the whole thing rather than renting it. Odysseus is an agent harness, the same idea I broke down in.

When the biggest channel on earth builds its own, you know where this is going.

So here's how to setup your first local LLM this weekend.

Local models are insurance. You keep using frontier models for the heavy lifting. You keep a local fallback for the day access vanishes, or you need privacy, or the wifi drops on a train and you still want to work.

I call the setup the Local Stack. Eight steps, in order.

1. Pick your runtime (Ollama or LM Studio).
2. Match the model to your hardware.
3. Pick the right model for the job.
4. Shrink it with quantisation.
5. Point at your local model.
6. Mind the context window.

7. Give the model tools.

8. Fine-tune later.

What you need

- A laptop (a Mac with 16GB or more of memory is ideal)
- Ollama or LM Studio, both free
- One model downloaded to start (a small one)
- Twenty minutes

The first five steps get you running. The last three make you good at it.



How to Run Claude Code for Free

Set up a local LLM and run it **free, private, offline**. charliehills.substack.com

[START HERE](#)

8GB+ RAM · works on any Mac or PC

Install a runtime

THE ENGINE

The free app that runs models on your computer. Pick one:



Ollama TERMINAL

CLI + local API. Get it at ollama.com.

Best for control and speed.



LM Studio NO CODE

A full app to browse, download and chat. lmstudio.ai.

Best for beginners.

Download a model

ONCE, OFFLINE

- 1 Open **Terminal** (Mac) or **PowerShell** (Windows), free and already on your computer.
- 2 Type this line, then press Enter:

```
ollama pull qwen3-coder
```

It downloads **once**, then it is yours offline forever. Smaller models download faster.

Launch on your model

FULLY OFFLINE

```
ollama launch claude
```

- A menu pops up. Arrow-key to your model, press Enter. **Done.**



qwen3-coder
best first pick



deepseek
reasoning

What you get

THE PAYOFF

- ✓ **Free.** Runs on your own model, no Anthropic bill.
- ✓ **Private.** Nothing leaves your computer, works offline.
- ✓ **Yours forever.** No usage limits, no monthly cap.

Pick your model

SIZE · RAM



Gemma 3

tiny, runs on a phone

~1GB

8GB



Llama 3.2

fast all-purpose

~2GB

8GB



Qwen 3

best all-rounder

~5GB

16GB



DeepSeek

reasoning & coding

~5GB

16GB



Mistral

fast & efficient

~4GB

16GB



Qwen 32B

power tasks

~20GB

32GB



Llama 70B

top quality

~43GB

64GB

model size ≈ **RAM needed**. Q4 = 4-bit, ~95% quality (the default).

Install Claude Code

NO ANTHROPIC BILL

MAC / LINUX

```
curl -fsSL https://claude.ai/install.sh | bash
```

WINDOWS

```
irm https://claude.ai/install.ps1 | iex
```

Takes about two minutes. When it says **"Installed"**, you are ready.

Now build anything

PLAIN ENGLISH

Build me a website for my business

Turn my idea into a working app

Find and fix the bug in my code

✓ Reads files ✓ Writes files ✓ Runs code

Handy commands

CHEAT SHEET

`ollama list` see your models

`/help` all commands

`/compact` free up context

`ollama pull` get a model

`/init` learn your project

`/clear` fresh chat, keep memory

`ollama rm` delete a model

`/model` switch the model

`/rewind` undo a change



CHARLIE HILLS · charliehills.substack.com

If you want the secret sauce behind how I generate GIF infographics like this, [apply here](#).

Step one → Pick your runtime

A runtime is the program that runs a model on your own computer. It's the engine. No runtime = no local AI.

You have two free options. Ollama is the one I use. Download it from ollama.com, install it like any app, and it quietly runs in the background.

Download Ollama



```
irm https://ollama.com/install.ps1 | iex
```



paste this in PowerShell

or

Download for Windows

Requires Windows 10 or later

LM Studio is the other, with a friendlier window if you'd rather click than type.



Once Ollama is installed it serves models at a local address on your machine, `localhost:11434`. You don't need to understand that line to use it. Just know it's the door everything else knocks on later.

To download your first model, you run one line:

```
ollama pull [model]
```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\91969> ollama --version
ollama version is 0.30.10
PS C:\Users\91969> ollama pull qwen3
pulling manifest
pulling a3de86cd1c13: 100% 5.2 GB
pulling ae370d884f10: 100% 1.7 KB
pulling d18a5cc71b84: 100% 11 KB
pulling cff3f395ef37: 100% 120 B
pulling 05a61d37b084: 100% 487 B
verifying sha256 digest
writing manifest
success
PS C:\Users\91969>
```

That pulls the model onto your hard drive. From then on it runs offline.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\91969> ollama --version
ollama version is 0.30.10
PS C:\Users\91969> ollama run qwen3
>>> Explain what Ollama does in two sentences.
PS C:\Users\91969> ollama run qwen3
>>> Why does running an AI model locally matter?
Thinking...
Okay, so the user is asking why running an AI model locally matters. Let me break this down. First, I need to recall what "running an AI model locally" means. It means deploying the model on a local device, like a computer or a smartphone, rather than on a remote server or cloud platform.

Hmm, why would someone want to do that? Well, there are a few key reasons. Let me think. Data privacy is one reason. If you run the model locally, the data doesn't have to leave the device. That's important for sensitive information like medical records or personal messages.

Then there's latency. If the model is on a local device, processing is faster because there's no need to send data over the internet. That's crucial for real-time applications like self-driving cars or voice assistants.

Bandwidth usage is another factor. If you're using a lot of data, running models locally reduces the amount of data transmitted, which can be beneficial for users with limited data plans or in areas with poor internet connectivity.

Energy efficiency might be a consideration too. Cloud computing can be energy-intensive, so running models locally might be more environmentally friendly, especially for devices with limited power.

Customization and control are also important. Local models can be tailored to specific hardware, making them more efficient. Plus, users have more control over the model's behavior and data handling.

Legal and compliance issues come into play as well. Some industries have strict regulations about data handling, and running models locally can help ensure compliance.
```

Tip: Start with one small model before you download a 40GB monster you'll never use. You can always pull more.

Step two → Match the model to your hardware

A model's size is measured in billions of parameters. You'll see numbers like 7B, 27B, 70B. Bigger is smarter, and bigger needs more memory.

qwen3

↓ 31M ⌚ 8 months ago

Qwen3 is the latest generation of large language models in Qwen series, offering a comprehensive suite of dense and mixture-of-experts (MoE) models.

tools

thinking

0.6b

1.7b

4b

8b

14b

30b

32b

235b

Here's the rough rule:

- A 7B model runs on almost any laptop
- A 27B to 32B model wants 32GB or more of memory
- A 70B model needs serious hardware, like a maxed-out Mac Studio

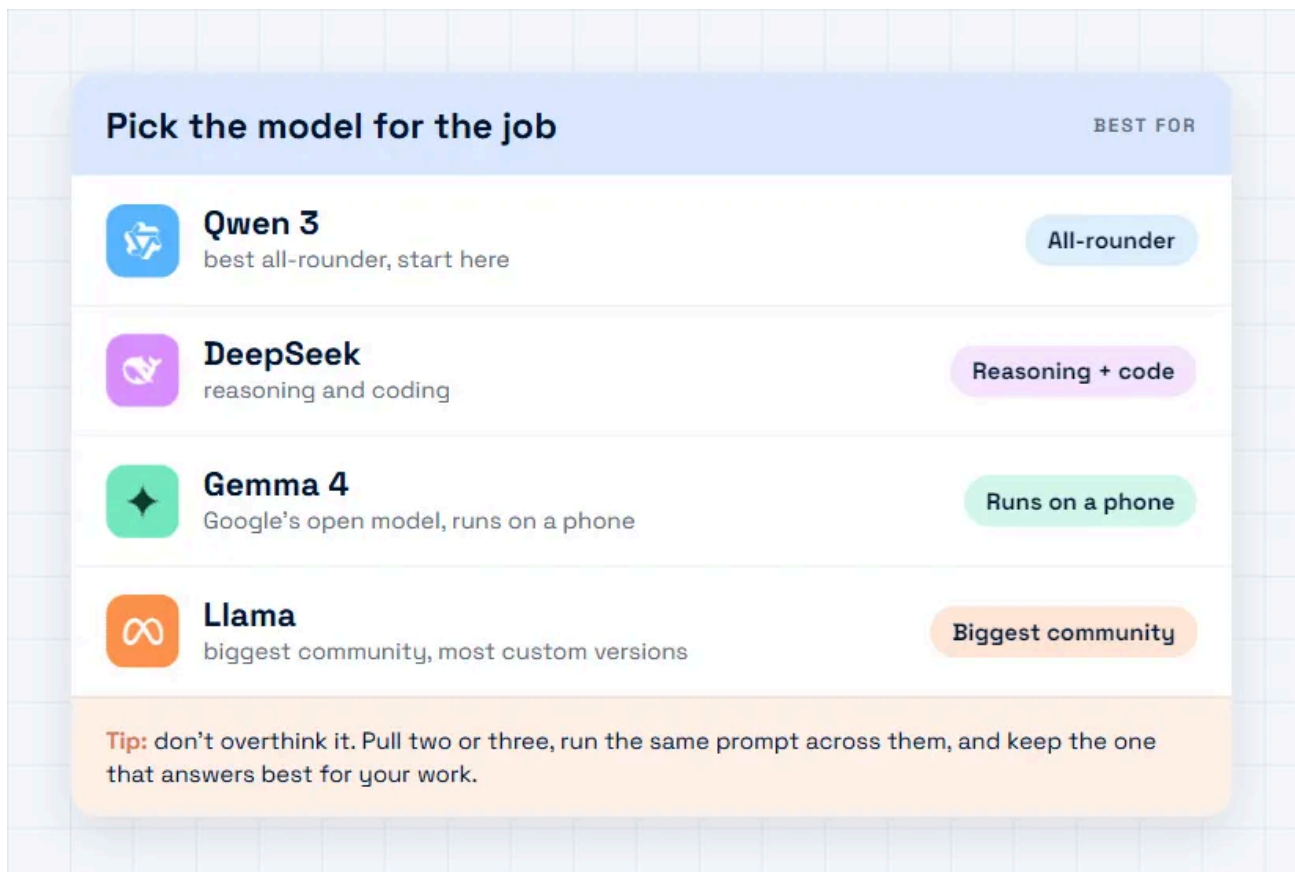
If a model is too big for your memory, it spills over and runs three to five times slower. So match the model to the machine you actually own, not the one you wish you had.

Use this step when: you're about to download a model and you're not sure your laptop can handle it. Check the size first.

Step three → Pick the right model for the job

Different open models are good at different things. These four cover almost everything a marketer or operator needs:

- **Qwen 3** is the best all-rounder for most tasks.
- **DeepSeek** is the one for reasoning and coding.
- **Gemma 3** is tiny enough to run on a phone.
- **Llama** has the biggest community.



You're not locked in. Pull two or three, test the same prompt across them, keep the one that answers best for your work.

Tip: The "best" model on a benchmark is rarely the best model for you. The right model is the one that runs smoothly on your machine and gives you usable answers. A fast 7B you actually use beats a 70B you never load.

Step four → Shrink it with quantisation

This is the one concept that changes everything, and almost no one outside the field knows it.

Quantisation shrinks a model so it runs on weaker hardware, with barely any drop in quality. When you browse models you'll see versions labelled Q4 or Q5. Those are the shrunk ones. Q4 keeps roughly 95% of the quality at a fraction of the memory.

qwen3:235b 754a872f1290 · 8 months ago	142GB	256K	Text
qwen3:0.6b-q4_K_M 7df6b6e09427 · 1 year ago	523MB	40K	Text
qwen3:0.6b-q8_0 f6358994266d · 1 year ago	832MB	40K	Text
qwen3:0.6b-fp16 626c9556a80f · 1 year ago	1.5GB	40K	Text
qwen3:1.7b-q4_K_M 8f68893c685c · 1 year ago	1.4GB	40K	Text
qwen3:1.7b-q8_0 29b6a7a420a4 · 1 year ago	2.2GB	40K	Text
qwen3:1.7b-fp16 e9354f6174db · 1 year ago	4.1GB	40K	Text
qwen3:4b-instruct 0edcdef34593 · 8 months ago	2.5GB	256K	Text

This is how a model that “needs” a server ends up running fine on your laptop. Look for the Q4 version by default. Learn this word and the whole thing opens up.

Step five → Point Claude Code at your local model

This is the payoff, and it’s the part I verified myself across all those versions.

Claude Code is the terminal tool a lot of us now build with. By default it talks to Anthropic’s cloud. You can point it at a model running on your own machine instead.

You don’t need to touch a config file. With Ollama installed, one line does the whole thing:

```
ollama launch claude
```

A menu pops up. Arrow-key to your local model, press Enter. Behind the scenes Ollama points Claude Code at your machine instead of the cloud, so your requests never leave your laptop. Now the tool runs on a local model. Free. Offline. Yours.

```
Windows PowerShell
Select model for Claude Code: Type to filter...

Recommended
kimi-k2.7-code:cloud
  Coding-focused long-horizon agentic workflows with improved token efficiency and vision
glm-5.2:cloud
  Long-horizon coding and agentic engineering with a solid 1M context
minimax-m3:cloud
  State-of-the-art coding & agent capabilities with multimodal reasoning and support for up to 1M context
nemotron-3-super:cloud
  Reasoning, coding, and agentic tool use
gemma4
  Agentic workflows and multimodal reasoning, ~12GB, (not downloaded)
qwen3.6
  Agentic software engineering model with multimodal capabilities, ~28GB, (not downloaded)

More
  qwen3.6

↑/↓ navigate • enter select • ← back
```

Tip: you'll want a recent Ollama (v0.15 or later) for the launch command. Update first if ollama launch isn't recognised.

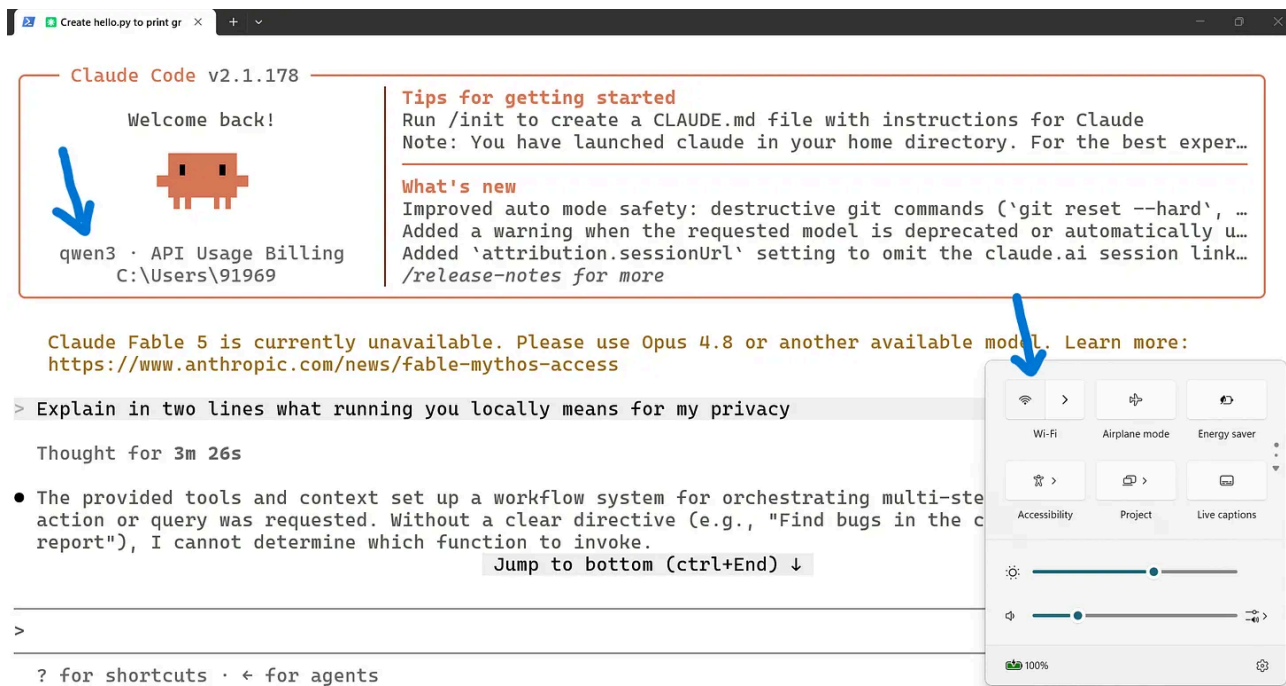
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\91969> ollama --version
ollama version is 0.30.10
PS C:\Users\91969> |
```

Want more than Ollama's one-liner? There's a popular open-source proxy called free-claude-code (<https://github.com/Alishahryar1/free-claude-code>), 35k stars and counting, that points Claude Code at 17+ backends, local models or free API tiers, so you can swap which model sits behind Claude Code without touching a config file.

I won't oversell it. A local 7B won't match a frontier model on your hardest problems, and I'll come back to that. But for a huge share of everyday work, a tuned local coder model is plenty. And on the Friday a model gets pulled, this is the difference between your workflow stopping and your workflow carrying on.

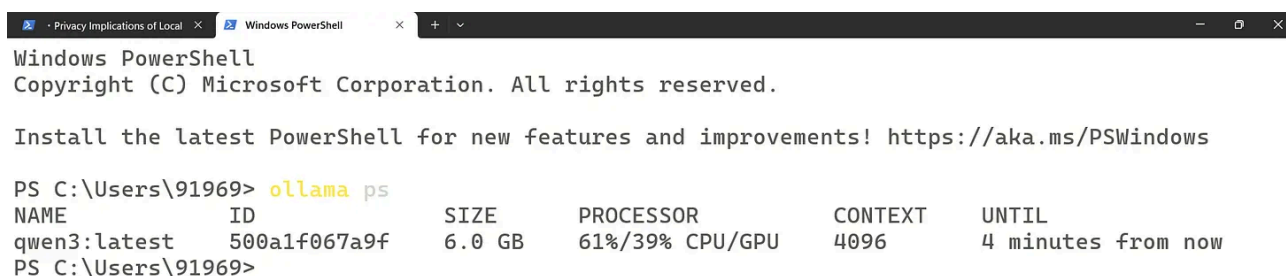


Use this step when: you want a fallback that survives an outage, a ban, or a flight with no wifi.

Step six → Mind the context window

The context window is how much the model can hold in its head at once. In the cloud you get a huge one for free. Locally, you pay for it in memory.

A bigger context window eats memory fast, and a model that was running smoothly will start to choke. So keep your sessions tight and your prompts lean. Clear the history when you move to a new task. This is the trade you make for running on your own machine, and once you feel it, you manage it without thinking.

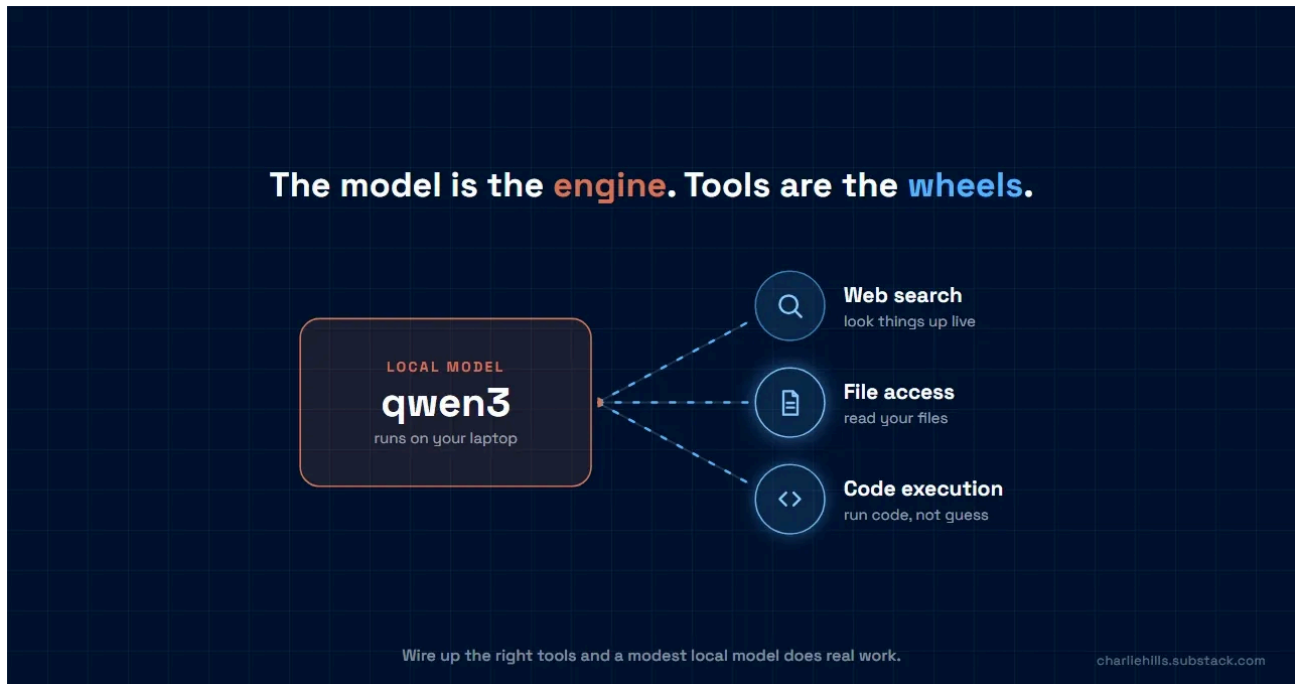


Step seven → Give the model tools

A smaller local model with web search, file access and code execution beats a giant model with none.

The model is the engine. Tools are the wheels. Wire up the right ones and a modest local model handles real work, because it can look things up, read your files and run code rather than guessing from memory alone.

The capability gap between local and frontier closes fast the moment you add tools.



Tip: Before you chase a bigger model, give your current one tools. It's usually the faster win.

Step eight → Fine-tune later

You don't need this on day one. You should know it exists.

Fine-tuning means taking an open model and training it on your own data so it gets good at your specific work, your brand voice, your domain. It's more accessible than it sounds, and it's the reason owning an open model matters. You can shape it. You can't do that with a model you only rent.

Park this until the first five steps are second nature. Then come back to it.

The honest limitation

Local models are not a replacement for frontier models. They're insurance.

A local 7B won't match a Fable-5-class model on your hardest reasoning, your trickiest code, your longest documents. The models that get close need real hardware, the kind most people don't have at home yet. And context costs you memory every single session.

So this isn't a breakup with the cloud. I still reach for the best frontier model for the heavy work. The point is to never again have my whole workflow depend on access I don't control. Own one layer. Keep a local fallback ready. That's the insurance policy, and the premium is twenty minutes this weekend.

The move this week is small. Install Ollama. Pull one 7B model. Run one prompt offline. That's the whole policy bought.

The people who set this up before they need it will be the ones still working on the day the next letter lands.